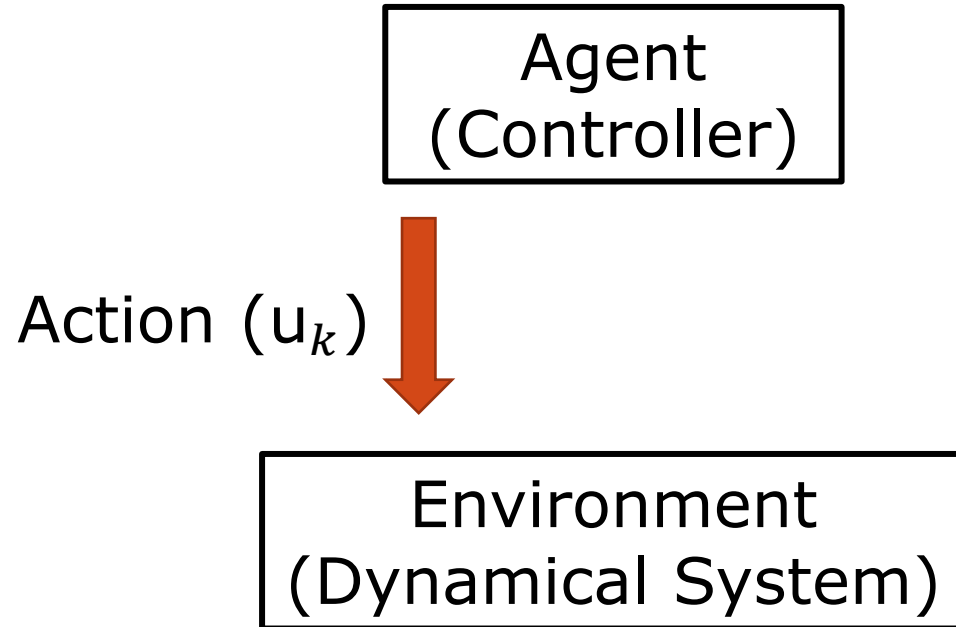


REINFORCEMENT LEARNING

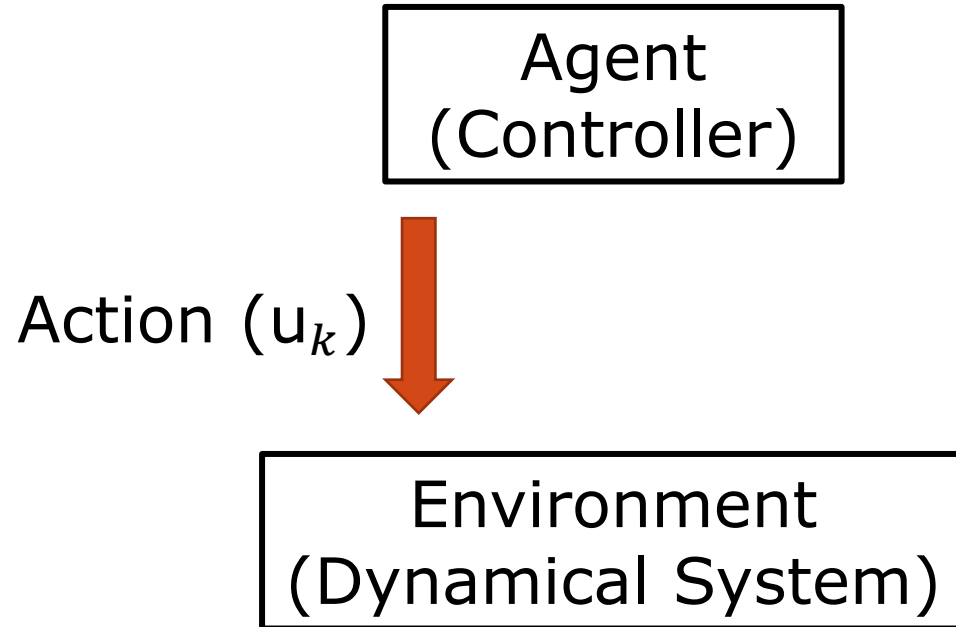
Agent
(Controller)

Environment
(Dynamical System)

REINFORCEMENT LEARNING

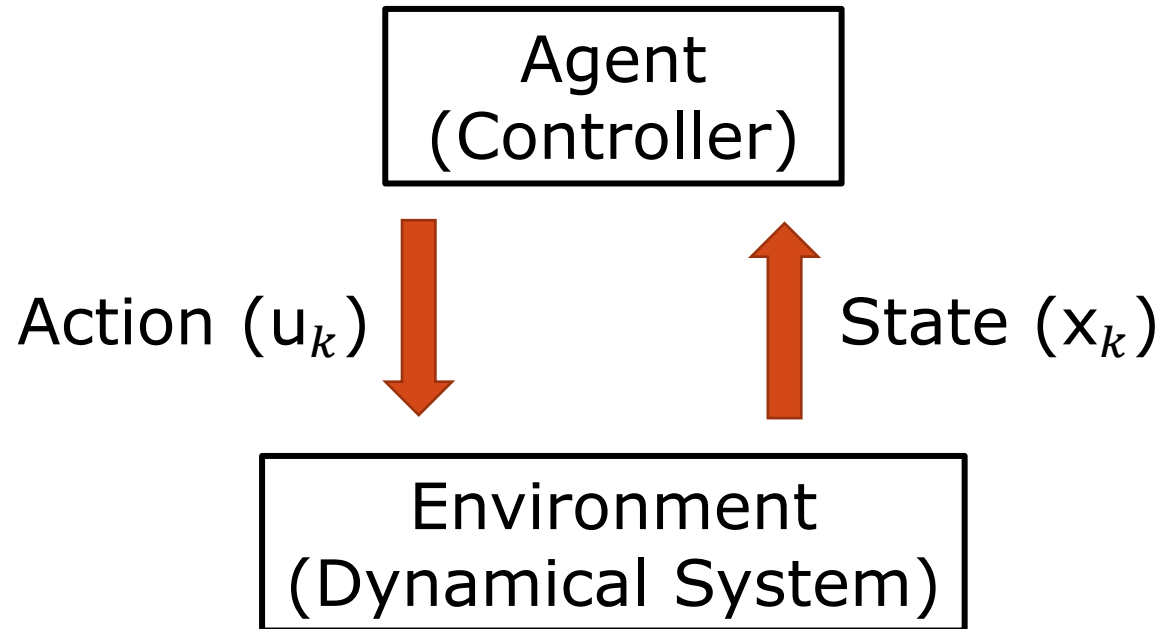


REINFORCEMENT LEARNING



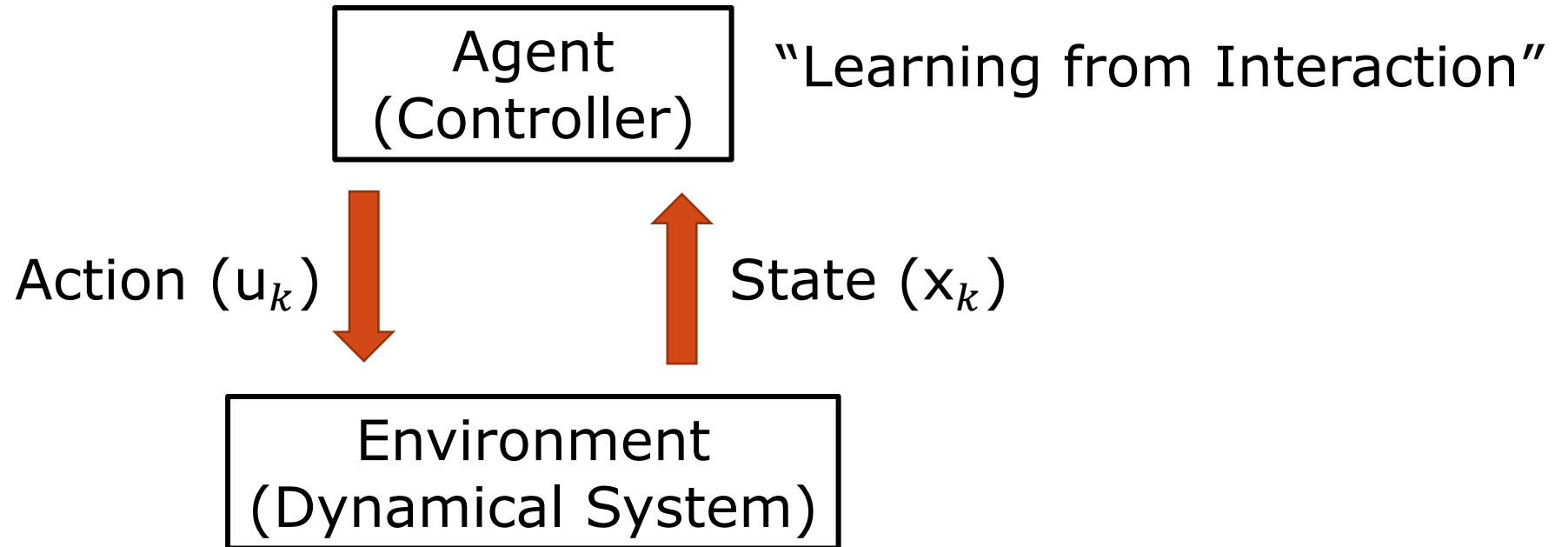
- $u_k, x_k \rightarrow r(x_k, u_k)$ (reward)
- $u_k, x_k \rightarrow x_{k+1}$ (next state)

REINFORCEMENT LEARNING



- $u_k, x_k \rightarrow r(x_k, u_k)$ (reward)
- $u_k, x_k \rightarrow x_{k+1}$ (next state)

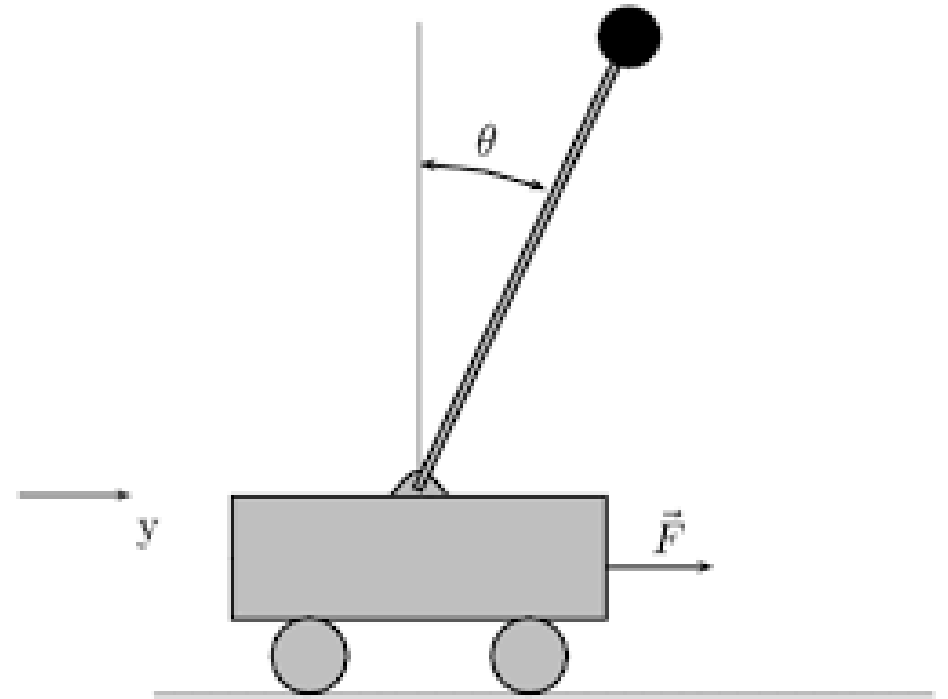
REINFORCEMENT LEARNING



- $u_k, x_k \rightarrow r(x_k, u_k)$ (reward)
- $u_k, x_k \rightarrow x_{k+1}$ (next state)

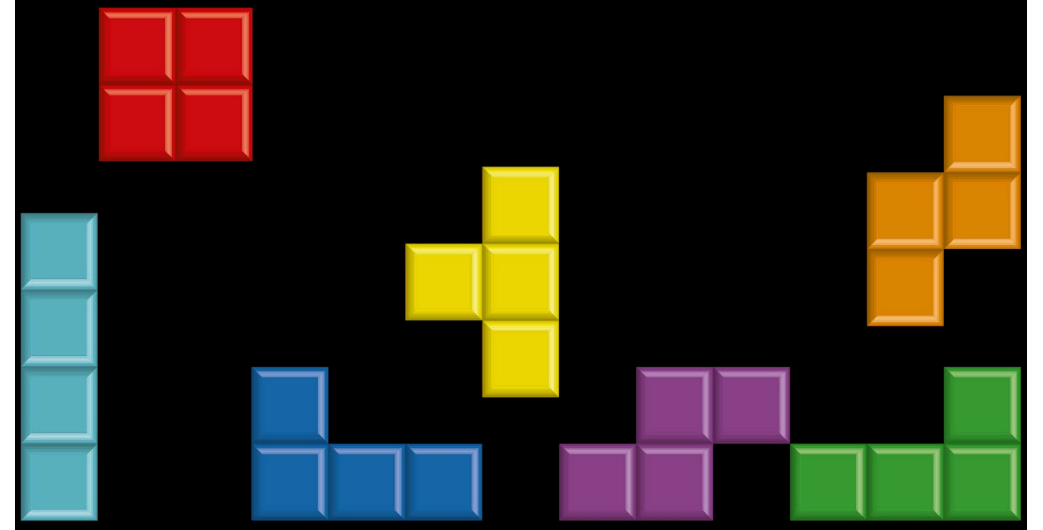
EXAMPLE: CART-POLE

- **Objective:** balance a pole on top of a movable cart
- **State:** angle, position, horizontal velocity, ...
- **Action:** horizontal force
- **Reward:** unit reward at each time step when the pole is upright



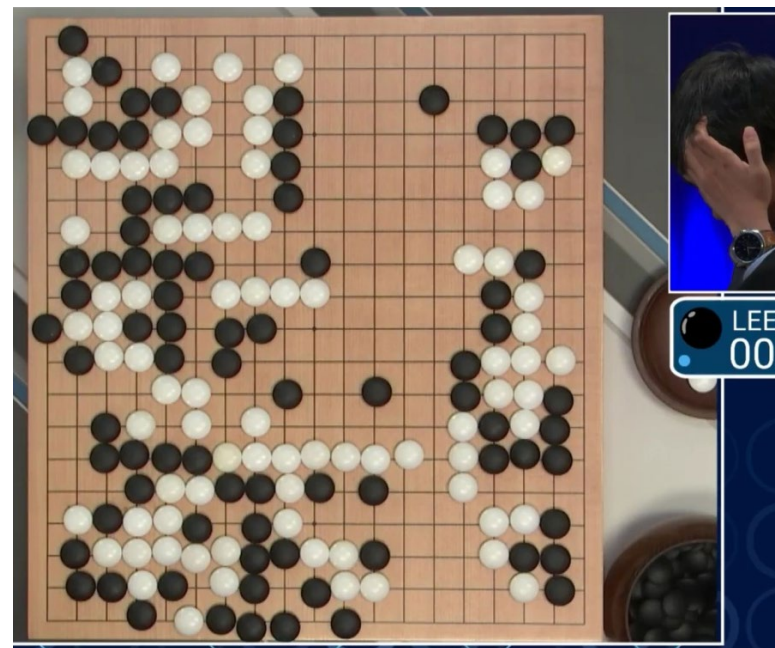
EXAMPLE: TETRIS

- **Objective:** stay in the game for as long as possible
- **State:** the state of the tiles
- **Action:** orientation of the falling piece
- **Reward:** unit reward at each time staying in the game



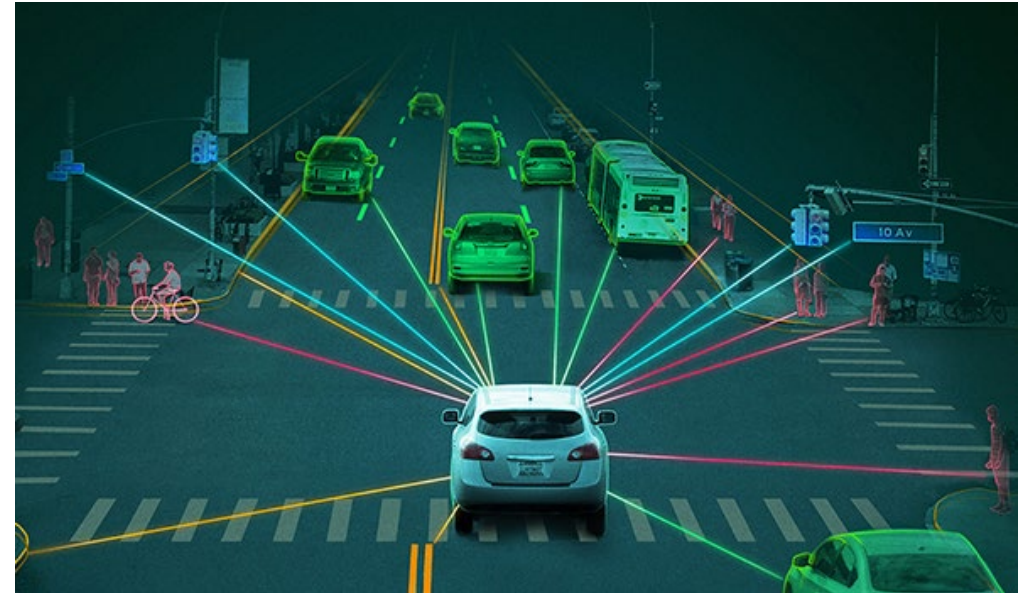
EXAMPLE: GO

- **Objective:** win the game
- **State:** position of all pieces
- **Action:** location of the next piece
- **Reward:** unit reward when winning the game and zero otherwise



EXAMPLE: SELF-DRIVING

- **Objective:** safe and efficient transportation
- **State:** position of the car and the surrounding environment
- **Action:** steering, brakes, accelerates
- **Reward:** reward when keeping in the lane, avoiding accident



REINFORCEMENT LEARNING

Agent



Action



State

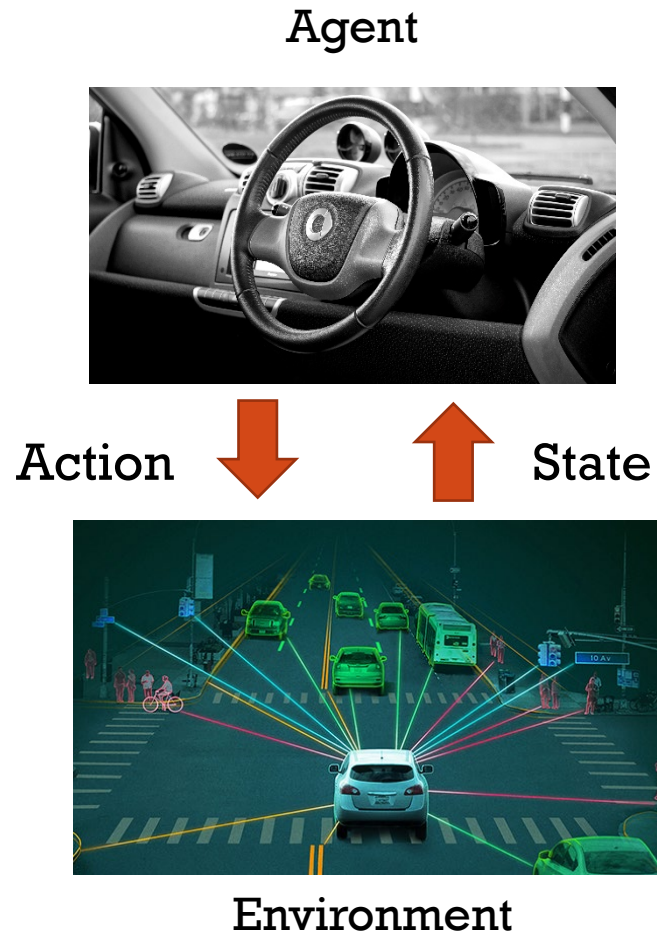


Environment

Important Features

- Discover optimal actions by trying them (trial-and-error search)
- Current action affects next state
- Maximize long-term return instead of instantaneous rewards

REINFORCEMENT LEARNING



Difference from supervised learning

- no access to labelled data (state, optimal action)
- RL: learning from its own experience (have to explore)

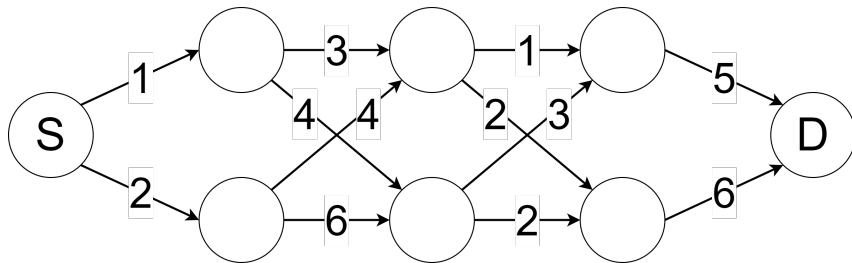
Difference from unsupervised learning

- Unsupervised learning: finding hidden structure (e.g. clustering using PCA or K-means)
- RL: maximizing the long-term return

RL: Data-driven decision/control engines (instead of knowledge engines)

Dynamic Programming (DP) and Principle of Optimality

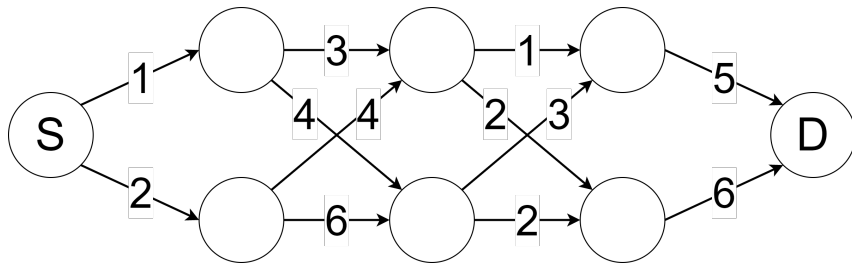
A Deterministic DP Example



Find the shortest path from S to D.

Exclusive search (forward search): number of possible paths = $2 \times 2 \times 2 = 8$

A Deterministic DP Example

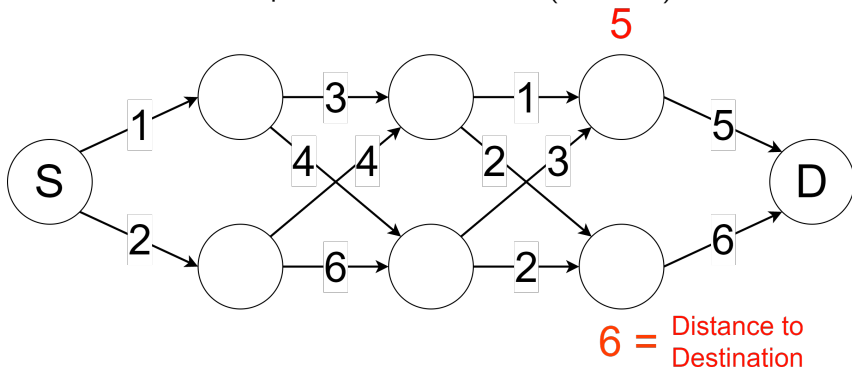


Find the shortest path from S to D.

DP (backward search): start from the destination

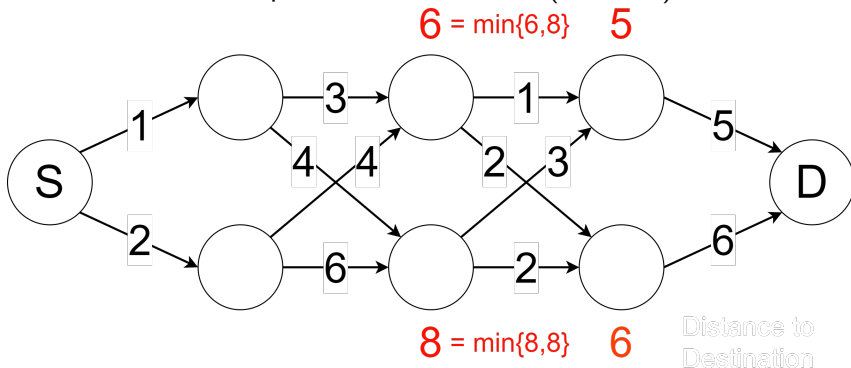
DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



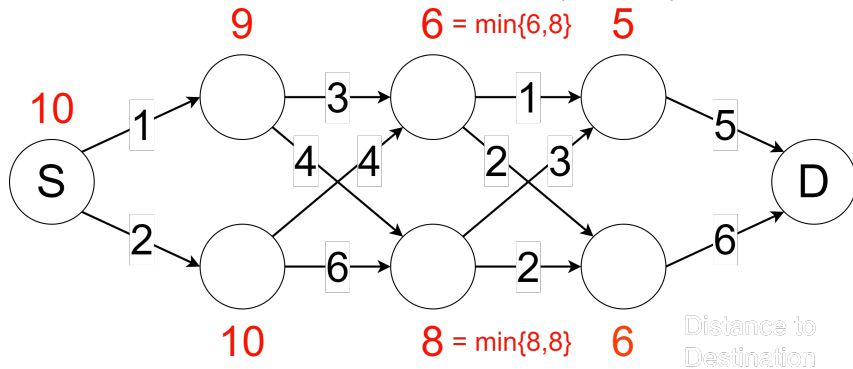
DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



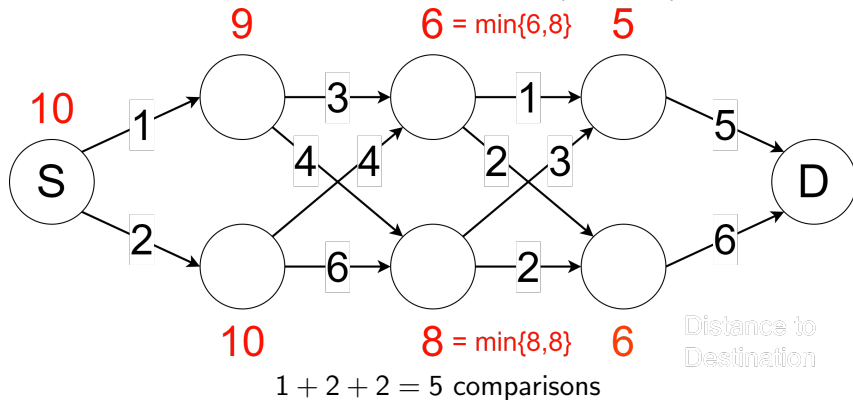
DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



DP: backward search

- Calculate the shortest distance from a current node to destination D.
- Each calculation is a comparison of two choices (numbers)



Deterministic DP

The computational complexity of exclusive search versus DP.

Suppose the problem has T stages. Then

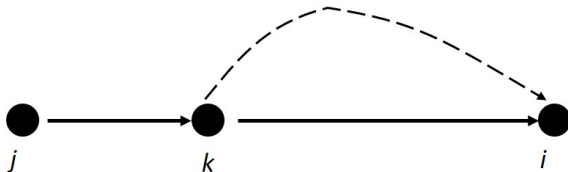
- Exclusive search: exponential in T
- Dynamic programming: linear in T

Principle of Optimality

A subsolution of an optimal solution is the optimal solution for the corresponding subproblem.

$$d(j, i) = \min_{k \in \mathcal{N}(j)} c(j, k) + d(k, i)$$

$\mathcal{N}(j)$ is set of outgoing neighbors of node j and $c(j, k)$ is the distance of edge (j, k) .



Finite horizon deterministic DP

System equation:

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, 1, \dots, N-1$$

x_k : system state at time k , u_k : control at time k

Finite horizon deterministic DP

System equation:

$$x_{k+1} = f_k(x_k, u_k), \quad k = 0, 1, \dots, N-1$$

x_k : system state at time k , u_k : control at time k

In the shortest path problem,

x_k : on the top or bottom node at stage k

u_k : go to the top or bottom node at stage $k+1$

Finite horizon deterministic DP

For given initial state x_0 , the cost over control sequence $(u_0, u_1, \dots, u_{N-1})$ is defined to be

$$J(x_0; u_0, \dots, u_{N-1}) = g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k)$$

Finite horizon deterministic DP

For given initial state x_0 , the cost over control sequence $(u_0, u_1, \dots, u_{N-1})$ is defined to be

$$J(x_0; u_0, \dots, u_{N-1}) = g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k)$$

Optimal cost and optimal control:

$$J^*(x_0) = \min_{u_0, \dots, u_{N-1}} J(x_0; u_0, \dots, u_{N-1})$$

Finite horizon deterministic problem

Backward computation:

$$J_N^*(x_N) = g_N(x_N). \quad (\text{trivial})$$

In the shortest path problem, we have

$$g_N(x_N) = 0.$$

Finite horizon deterministic problem

Backward computation:

$$J_N^*(x_N) = g_N(x_N). \quad (\text{trivial})$$

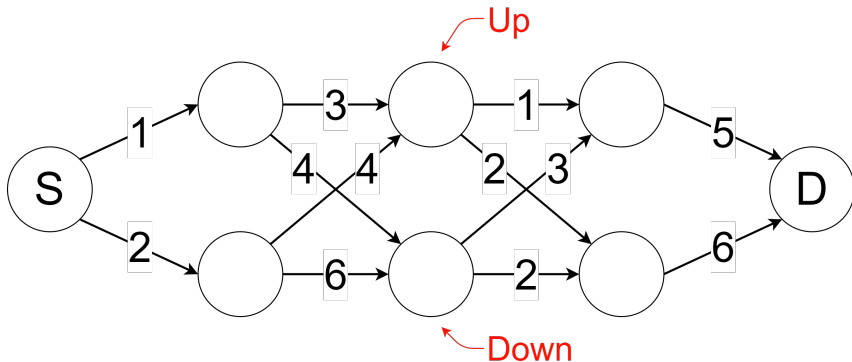
According to the DP principle,

$$J_{N-1}^*(x_{N-1}) = \min_{u_{N-1}} (g_{N-1}(x_{N-1}, u_{N-1}) + J_N^*(f(x_{N-1}, u_{N-1}))),$$

where we assume $f_k(\cdot) = f(\cdot)$ for all K . And in general,

$$J_k^*(x_k) = \min_{u_k} (g_k(x_k, u_k) + J_{k+1}^*(f(x_k, u_k))).$$

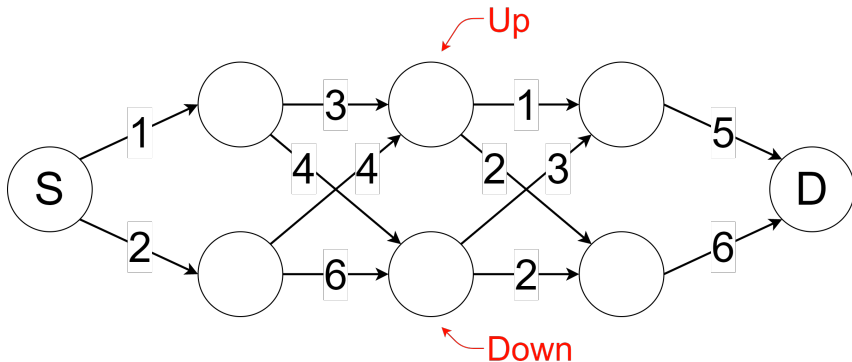
Finite horizon stochastic problem



$$u_k = \text{up} \rightarrow \begin{cases} P(x_{k+1} = \text{top}) = 0.6 \\ P(x_{k+1} = \text{bottom}) = 0.4 \end{cases}$$

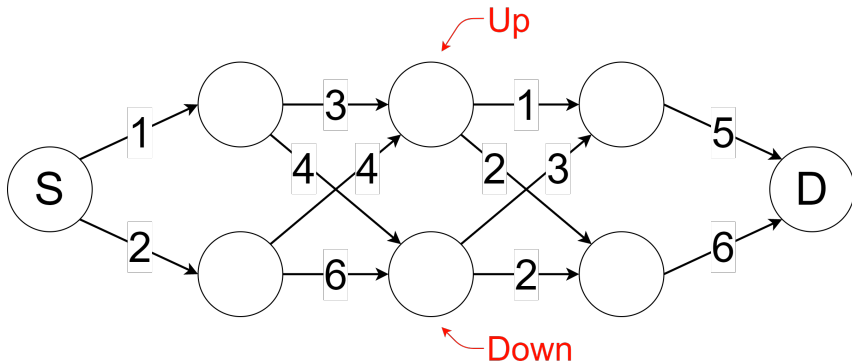
$$\text{and } u_k = \text{down} \rightarrow \begin{cases} P(x_{k+1} = \text{top}) = 0.4 \\ P(x_{k+1} = \text{bottom}) = 0.6 \end{cases}$$

Stochastic DP example



Optimal policy: A fixed sequence $(u_0, u_1, \dots, u_{N-1})$?

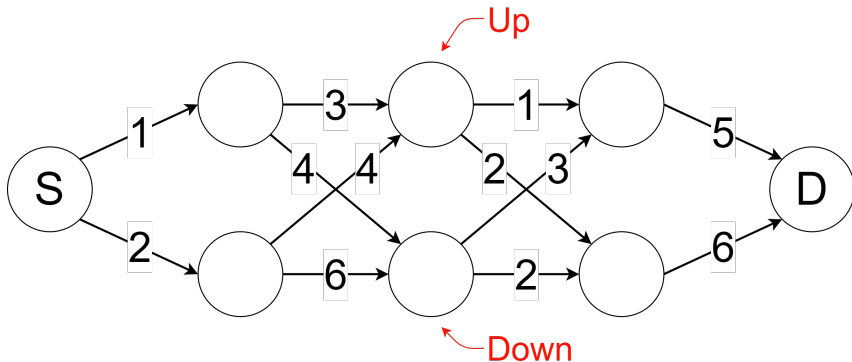
Stochastic DP example



Optimal policy: A fixed sequence $(u_0, u_1, \dots, u_{N-1})$?

NO.

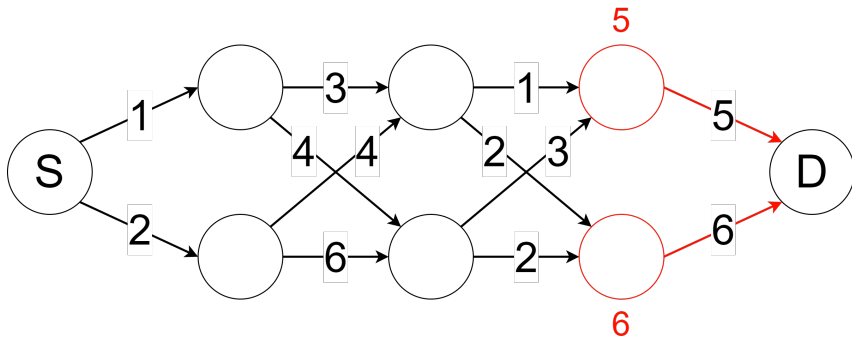
Stochastic DP example



Optimal policy: A fixed sequence $(u_0, u_1, \dots, u_{N-1})$?

State dependent policy: $\mu_k^*(x_k), \forall x_k$

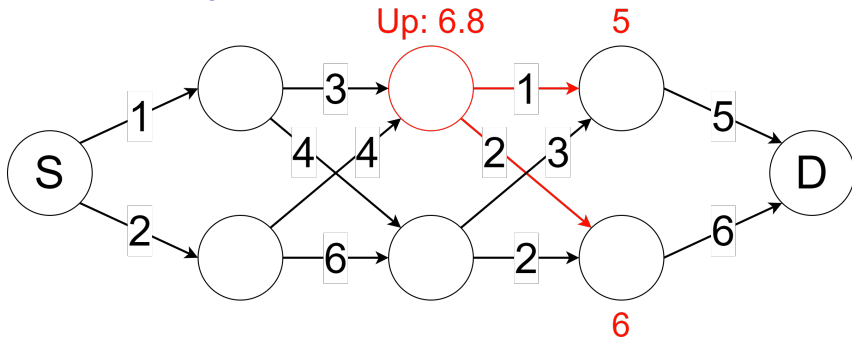
Stochastic DP example



The red nodes have only one possible path to reach node D.

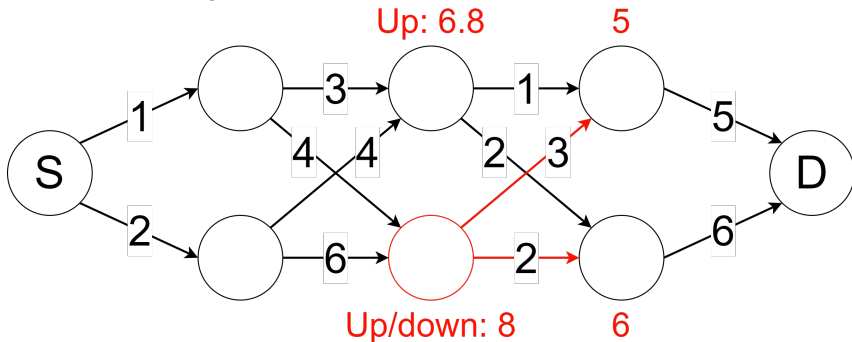
- Cost-to-go from the top node: 5
- Cost-to-go from the bottom node: 6

Stochastic DP example



- Cost-to-go with action Up: $0.6 \times (1 + 5) + 0.4 \times (2 + 6) = 3.6 + 3.2 = 6.8$
- Cost-to-go with action Down: $0.4 \times 6 + 0.6 \times 8 = 7.2$
- $\min\{\text{up}, \text{down}\} = 6.8 \implies u_{N-2}^*(\text{top}) = \mathbf{up}$

Stochastic DP example



- Cost-to-go (Up): $0.6 \times (3 + 5) + 0.4 \times (2 + 6) = 8$
- Cost-to-go (Down): $0.4 \times 8 + 0.6 \times 8 = 8$
- $\min\{\text{up}, \text{down}\} = 8 \implies u_{N-2}^*(\text{bottom})$ can be either up or down

Stochastic DP

- Random transitions $P(x_{k+1}|x_k, u_k)$
- Cost function:

$$E \left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k) \right]$$

- Given initial state x_0 , find a policy

$$\pi = \{\mu_0, \mu_1(x_1), \dots, \mu_{N-1}(x_{N-1})\}$$

that minimizes

$$J(x_0) = \min_{\pi} E \left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k)) \right]$$

Stochastic DP

The Bellman Equation:

$$\begin{aligned} J_k^*(x_k) &= \min_{u_k} E [g_k(x_k, u_k) + J_{k+1}^*(x_{k+1})] \\ &= \min_{u_k} \sum_g gP(g_k = g|x_k, u_k) + \sum_x J_{k+1}^*(x)P(x_{k+1} = x|x_k, u_k) \end{aligned}$$

Given state x_k and $J_{k+1}^*(x_{k+1})$,

$$u_k^* \in \arg \min_{u_k} E[g_k(x_k, u_k) + J_{k+1}^*(x_{k+1})]$$

Backward-forward algorithm: First compute J_k^* backward and then find u_k^* forward.

Stochastic Control versus Reinforcement Learning

- Transition kernel $P(x_{k+1}|x_k, u_k)$
- **Cost** function: $g_k(x_k, u_k)$
- Given initial state x_0 , find a policy μ such that

$$J_0(x_0) =$$

$$\min_{\mu} E \left[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k)) \right]$$

- Transition kernel $P(x_{k+1}|x_k, u_k)$
- **Reward** function: $r_k(x_k, u_k)$
- Given initial state x_0 , find a policy μ such that

$$V_0(x_0) =$$

$$\max_{\mu} E \left[r_N(x_N) + \sum_{k=0}^{N-1} r_k(x_k, \mu_k(x_k)) \right]$$

Infinite Horizon Discounted RL

Time-homogeneous stochastic systems (environments)

- Transition kernel $P(x_k = j | x_{k-1} = i) = P(x' = j | x = i)$ does not depend on k
- Reward function $r_k(x_k = x, u_k = u) = r(x, u)$ does not depend on k
- Discount factor $0 < \gamma < 1$ and initial state x_0 , find a policy μ such that

$$V_0(x_0) = \max_{\mu} E \left[\sum_{k=0}^{\infty} \gamma^k r(x_k, \mu_k(x_k)) \right]$$

The Bellman Equation

$$\begin{aligned} V_k^*(x_k) &= \max_{\mu_k} E [r(x_k, \mu_k(x_k)) + \gamma V_{k+1}^*(x_{k+1})] \\ \implies V^*(x) &= \max_{\mu} E [r(x, \mu(x)) + \gamma V^*(x')] \end{aligned}$$

The Bellman Equation in RL

Episodic RL

$$V_k^*(x_k) = \max_{\mu_k} E [r_k(x_k, \mu_k(x_k)) + V_{k+1}^*(x_{k+1})]$$

Infinite Horizon Discounted RL

$$V^*(x) = \max_{\mu} E [r(x, \mu(x)) + \gamma V^*(x')]$$

The goal of reinforcement learning: Learn the optimal policy μ^* by solving the Bellman equation.